

Shell Scripting: Functions & intermediate concepts

❖ Task 1: Basic Functions

- A function greet that takes a name as argument and prints Hello, <name>!
- A function add that takes two numbers and prints their sum
- Call both functions from the script

- `#!/bin/bash`
- `# Function to greet a user`
- `greet() {`
- `local name=$1`
- `echo "Hello, $name"`
- `}`
- `# Function to add two numbers`
- `add() {`
- `local num1=$1`
- `local num2=$2`
- `local sum=$((num1 + num2))`
- `echo $sum`
- `}`
- `greet "Dosto!"`
- `add 25 35`

```

ubuntu@ip-172-31-47-149:~$ vim funtions.sh
ubuntu@ip-172-31-47-149:~$ chmod 764 funtions.sh
ubuntu@ip-172-31-47-149:~$ ls -l funtions.sh
-rwxrw-r-- 1 ubuntu ubuntu 249 Feb 19 07:51 funtions.sh
ubuntu@ip-172-31-47-149:~$ ./funtions.sh
Hello, Dosto!
60

```

❖ Task 2: Functions with Return Values

- Create disk_check.sh with:
- A function check_disk that checks disk usage of /using df -h
- A function check_memory that checks free memory using free -h
- A main section that calls prints the results
 - check_disk () {
 - echo "--- Disk Usage ---"
 - df -h
 - }
 - # Function to check memory Usage
 - check_memory () {
 - echo "--- Memory Usage ---"
 - free -h
 - }
 - # Calling both functions
 - main() {
 - echo "System Resource Check"
 - echo "-----"

- check_disk
- check_memory
- }
- # Execute main function
- main

```

ubuntu@ip-172-31-47-149:~$ vim disk_check.sh
ubuntu@ip-172-31-47-149:~$ ./disk_check.sh
System Resource Check
-----
--- Disk Usage ---
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        19G   2.6G   16G   15% /
tmpfs            458M    0   458M    0% /dev/shm
tmpfs            183M   912K   182M    1% /run
tmpfs            5.0M    0    5.0M    0% /run/lock
efivarfs         128K   3.8K   120K    4% /sys/firmware/efi/efivars
/dev/nvme0n1p16  881M   89M   730M   11% /boot
/dev/nvme0n1p15  105M   6.2M   99M    6% /boot/efi
tmpfs            92M    12K   92M    1% /run/user/1000
--- Memory Usage ---
              total        used        free      shared  buff/cache
available
Mem:          914Mi        403Mi        181Mi        2.7Mi        487Mi
510Mi
Swap:          0B          0B          0B

```

❖ Task 3: Strict Mode – set –euo pipefail

- Create strict_demo.sh with set –euo pipefail at the top
- Try using an undefined variable – what happens with set –u?
- Try a command that fails –what happens with set –e?
- Try a piped command where one part fails –what happens with set –o pipefail?
 - set -euo pipefail
 - echo "Value of UNDEFINED_VAR is: \$UNDEFINED_VAR"
 - #echo "2) Demonstrating set -e (command failure)"
 - #false

- #echo "3) Demonstrating set -o pipefail (pipeline failure)"
- #echo "hello" | grep "nomatch" | wc -l

```
ubuntu@ip-172-31-47-149:~$ ./strict_demo.sh
./strict_demo.sh: line 9: UNDEFINED_VAR: unbound variable
```

Flag	Meaning
Set – e	Exit immediately if any command fails.
Set – u	Exit if using an undefined variable.
Set – o pipefail	If any command in a pipe fails, whole pipe fails

Document: What does each flag do?

- Set – e: Script stop if the variable is undefined.
- Set – u: Script exits when a command returns non-zero.
- Set – o: pipefail: Even if the last pipe command succeeds, failure in first part stops script.

❖ Task 4: Local Variables

- Create local_demo.sh with:
 - global_var="I am global"
 - test_local() {
 - local local_var="I am local"
 - echo "\$local_var"
 - }

- test_global() {
- temp_var="I leak outside"
- }
- test_local
- echo "\$local_var" # Will NOT print anything
- test_global
- echo "\$temp_var" # Will print value

```
ubuntu@ip-172-31-47-149:~$ vim local_demo.sh
ubuntu@ip-172-31-47-149:~$ ./local_demo.sh
I am local

I leak outside
```

❖ Task 5: Build a script – System Info Reporter

- Create system_info.sh that uses functions for everything:
- A function to print hostname and OS info
- A function to print uptime
- A function to print disk usage (top 5 by size)
- A function to print top 5 CPU- consuming processes
- A main function that calls all of the above with section headers
- Use set -euo pipefail at the top
 - #!/bin/bash
 - set -euo pipefail
 - print_header() { echo "" echo "\$1" echo "" }
 - system_info() { print_header "System Information" echo "Hostname: \$(hostname)" echo "OS: \$(uname -a)" }
 - uptime_info() { print_header "Uptime" uptime }

- `disk_info() { print_header "Disk Usage (Top 5)" df -h | head -n 6 }`
- `memory_info() { print_header "Memory Usage" free -h }`
- `cpu_info() { print_header "Top 5 CPU Processes" ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -n 6 }`
- `main() { system_info uptime_info disk_info memory_info cpu_info }`
- Main

```
ubuntu@ip-172-31-47-149:~$ vim system_info.sh
ubuntu@ip-172-31-47-149:~$ ./system_info.sh
```

System Information

```
Hostname: ip-172-31-47-149
OS: Linux ip-172-31-47-149 6.14.0-1018-aws #18~24.04.1-Ubuntu SMP Mon Nov 24 19:46:27 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
```

Uptime

```
14:10:33 up 1:29, 2 users, load average: 0.00, 0.00, 0.00
```

Disk Usage (Top 5)

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/root	19G	2.6G	16G	15%	/
tmpfs	458M	0	458M	0%	/dev/shm
tmpfs	183M	904K	182M	1%	/run
tmpfs	5.0M	0	5.0M	0%	/run/lock
efivarfs	128K	3.8K	120K	4%	/sys/firmware/efi/efivars

Memory Usage

	total	used	free	shared	buff/cache	available
Mem:	914Mi	390Mi	167Mi	2.7Mi	514Mi	523Mi
Swap:	0B	0B	0B			

Top 5 CPU Processes

PID	PPID	CMD	%MEM	%CPU
1685	1630	sshd: ubuntu@pts/0	0.8	0.0
618	1	/usr/bin/containerd	5.2	0.0
556	1	/snap/snapd/current/usr/lib	4.1	0.0
1377	1322	sshd: ubuntu@pts/1	0.8	0.0
554	1	/snap/amazon-ssm-agent/1232	2.1	0.0

- ❖ What you learned
 - Functions make scripts modular and reusable
 - Local prevent variable leakage
 - Set `--euo` pipefail make script production-safe
- ❖ Explanation of set `--euo` pipefail:

- Is a common shell command used in bash scripts to enforce strict error handling
- - e: Exits the script immediately if any command fails
- - u: Treats unset variables as an error and exits the script.
- - o pipefail: Ensure that a pipeline. Returns the exit status of the first failed command, not just the last one
- Together these options help prevent silent failures and make scripts more reliable.